Policy-Pilot

Comprehensive Project Overview

Generated 17 August 2026 · Internal architecture brief

> Policy-Pilot is an AI-assisted access-request system. It takes a self-service request, grounds a recommendation in policy PDFs via RAG, and shows a structured **APPROVE | DENY | ESCALATE** result to a human. The model never grants or revokes access. The load-bearing rule is: **LLM suggests, human approves, deterministic code executes.**

1. Purpose

Someone asks for a system entitlement (for example `FIN_DATASET_READ` on `DATA_WAREHOUSE`). The backend then:

1. Embeds the request and searches policy chunks in Postgres (`pgvector`).
2. Asks an LLM for a Zod-validated recommendation with citations and a confidence score.
3. Stores the row as `PENDING_REVIEW` and writes an audit event.
4. Lets an admin approve, deny, or escalate in a React dashboard.
5. On approve, a **separate deterministic executor** (not the LLM) queues a grant, calls a mock downstream adapter, then upserts the entitlement.

2. Repository layout

npm workspaces on Node 22+, TypeScript 5.8 (strict, no `any`):

Path	Role
`apps/backend`	NestJS 11 API, BullMQ workers, RAG, Prisma
`apps/frontend`	React 19 + Vite + Tailwind HITL dashboard
`packages/shared-types`	Shared wire contracts (`PendingAccessRequest`, `AccessRecommendation`, …)
`shared/pii`	Recursive `maskPII` for logs and LLM observations
`evals/`	Golden dataset + `npm run eval` runner
`.cursor/rules/`	Architecture rules agents must follow
`docker-compose.yml`	Postgres 16 + pgvector, Redis 7

There is no root README; `AGENTS.md` and `docs/CODEBASE_REFERENCE.md` are the main guides. The reference doc is a snapshot and is behind the current tree in a few places (grant execution, evals, demo auth).

3. Tech stack

Layer	Choice
Language	TypeScript 5.8, strict mode, `no-explicit-any` as an ESLint error
Backend	NestJS 11, Zod 3 validation, Prisma 6
Database	PostgreSQL 16 with pgvector
Queue	BullMQ 5 on Redis 7, via `@nestjs/bullmq`
AI	OpenAI SDK (`text-embedding-3-small`, `gpt-4o-mini`), LangChain text splitters, pdf-parse
Frontend	React 19, Vite 6, Tailwind 3, TanStack Query 5, Axios
Testing	Jest 29 + ts-jest; Supertest for integration; Testing Library for React

4. How a request moves through the system

There are two ingest paths and one execution stage.

HITL (wired end-to-end)

A user submits title, department, cost center, system, entitlement, and justification. Retrieval builds a PII-safe query (no `employeeId`), embeds it, and takes the top 4 chunks. The decision engine returns JSON that must pass `RecommendationSchema`. The dashboard lists pending items with rationale, confidence, current entitlements, and policy citations. Admin actions update status, `decided_at`, and `access_audit_logs`.

Webhook

`POST /webhooks/access-requests` is idempotent, returns `202`, and enqueues work. The worker runs the same recommendation pipeline (`AccessRecommendationService.createFromWebhook`) rather than only hitting the mock downstream.

Grant

Approving sets `status = APPROVED` and `provisioning_status = QUEUED`, then enqueues a grant. The grant worker calls the downstream **before** upserting the entitlement, so a rate-limit rejection leaves no row. Overriding away from approve cancels a still-queued job, then revokes synchronously (revoke never calls downstream).

5. HITL HTTP surface

Method	Path	Who	Behavior
POST	`/access-requests`	user, admin	201 — generate recommendation and persist pending
GET	`/access-requests/pending`	admin	List `PENDING_REVIEW` rows
GET	`/access-requests/history`	admin	List decided items with provisioning status
POST	`/access-requests/:requestId/approve`	admin	Human approve; enqueue grant
POST	`/access-requests/:requestId/deny`	admin	Human deny
POST	`/access-requests/:requestId/escalate`	admin	Human escalate
POST	`/webhooks/access-requests`	upstream	202 — enqueue async recommendation job

Demo identity is sent as headers via `DemoAuthGuard` (roles `user` / `admin`), not real SSO.

6. AI / RAG subsystem

Policy PDFs in `apps/backend/data/policies/` are chunked (~1000/200 with heading detection), embedded with `text-embedding-3-small` (1536 dims), and stored in `policy_chunks`. Re-ingest deletes by `document_id` first.

Every chat call goes through `executeWithObservability`: load a versioned prompt from disk, mask PII for logs, time the call, parse JSON, `safeParse` against Zod, then emit prompt name/version, tokens, latency, estimated USD cost, and schema validity.

Prompts are never inlined. Active keys in `PROMPT_MANIFEST`:

- `system-policy` **v1.5.0** — grounding, verbatim citations, no `General` cites, mandatory ESCALATE on weak context, no-mutation rule, SoD vs dual-approval heuristics
- `rag-synthesis` v1.0.0 — template on disk (the engine currently uses `system-policy`)
- `eval-grounding-judge` **v1.1.0** — LLM-as-judge for evals (chunk support over golden paraphrase)

Structured output:

- `decision`: `APPROVE | DENY | ESCALATE`
- `rationale`
- `policy_citations` (`document_id`, `page_number`, `section_title`)
- `confidence_score` in `[0, 1]`

`DecisionEngineService` is injected only with `CHAT_CLIENT`. It has no Prisma or audit writer, so the model physically cannot mutate entitlements. Citations are also grounded against retrieved chunks after the LLM returns.

7. Data model

Table	Purpose
`users`	Hashed `employee_id` and `cost_center`; never raw IDs in that table
`entitlements`	Unique on `(userId, resourceName, permissionLevel)`
`access_requests`	HITL state, `recommendation_json`, `provisioning_status` (`NOT_APPLICABLE | QUEUED | PROVISIONED | FAILED`)
`access_audit_logs`	Append-only; PII-masked `previous_state` / `new_state`
`idempotency_keys`	Webhook, worker, and grant executions
`policy_chunks`	`vector(1536)`; Prisma treats this as Unsupported, so similarity search is raw SQL. No ANN index yet — exact scan at current corpus size.

8. Rate limits and queues

Two independent contracts, on purpose:

Domain	Queue	Ceiling	Worker
Ingest	`access-request-queue`	300 events / 60s	RAG recommendation
Execution	`access-grant-queue`	60 downstream calls / 60s	Grant adapter

Ingest can absorb a burst the downstream cannot. Approvals pile up in Redis instead of returning 500s. `MockDownstreamService` uses a global sliding window and reports `retryAfterMs`. The grant worker answers that with BullMQ `RateLimitError` so the job returns to `wait` **without** burning retry attempts.

9. Frontend

React dashboard with demo role switching (`user` / `admin`) and a requester profile gate:

Route	Who	What
`/profile`	anyone	Requester profile
`/`	user or admin	Submit a request
`/review`	admin	Pending queue, citations, approve / deny / escalate
`/history`	admin	Decided items plus provisioning badge

HTTP goes through TanStack Query hooks, not from components. `VITE_HITL_USE_MOCK_DATA` can serve in-memory data without NestJS.

10. Guardrails (non-negotiable)

1. **HITL isolation** — schema-validated recommendations only; grants run in a separate executor after a human decision.
2. **PII** — `maskPII` on `employee_id`, `cost_center`, `ssn`, `email` in logs, audit, and LLM observations. Retrieval omits `employeeId`.
3. **Zod at every boundary** — HTTP, repos, env, LLM output. Invalid model JSON becomes a 500, not a partial recommendation.
4. **Idempotency + audit** — webhooks, workers, and grants go through `idempotency_keys`; recommendation create and human decisions append audit rows.

11. Testing and evals

- **Unit:** Jest across backend, frontend, and `shared` (no infra required). `npm run verify` is format + lint + type-check + test, and is the Husky pre-commit gate.
- **Integration:** burst/idempotency against live Docker Postgres + Redis.
- **Evals:** `npm run eval` (`evals/run-evals.ts`) vs `evals/golden_dataset.json`. Tracks Recall@k, grounding (LLM-as-judge), citation hit rate, 100% schema validity, p95 latency, and token cost. Scenarios include happy path, SoD conflict, dual-approval escalate, and prompt-injection containment.

Latest eval scorecard

Run of 17 August 2026, 21:13 UTC against 15 golden cases. Prompts under test: `system-policy@1.5.0`, `rag-synthesis@1.0.0`, `eval-grounding-judge@1.1.0`. All 15 decisions matched the golden set. Gates: schema **PASS**, grounding **PASS** (threshold 0.80).

Metric	Result
Test cases	15
Recall@4	0.87
Grounding accuracy	87.33%
Schema validation	100%
Citation hit rate	1.00
p95 latency	3,704 ms
Estimated USD cost	$0.009436

Source: `evals/output/report.json`. Recall@4 is pulled down by TEST-09 and TEST-13 (named-section ranking misses those SoD chunks). Lowest grounding scores remain TEST-06, TEST-13, and TEST-15 (0.50).

12. Local stack

Typical bring-up:

1. `npm install`
2. Copy `.env.example` files and set `OPENAI_API_KEY`
3. `docker compose up -d`
4. Prisma migrate + seed
5. Ingest policy PDFs
6. Start NestJS backend and Vite frontend

Policy PDFs are gitignored, so a fresh clone has an empty RAG corpus until those files are supplied and ingested.

13. What is still incomplete

Relative to a production IAM product:

- Auth is **demo headers**, not real identity/SSO.
- Downstream is a **mock rate limiter**, not Okta / Workday / similar.
- Webhook `statusUrl` is advertised but not served.
- No ANN index on embeddings, and no CI workflow in-repo (quality gate is local Husky).
- `docs/CODEBASE_REFERENCE.md` still lists some gaps that the working tree has already closed (webhook → AI, post-approval grants, eval harness).

The current branch is a complete demo of the architecture: ingest → RAG → structured recommendation → human decision → queued deterministic grant, with PII masking, idempotency, and audit on the critical paths.

Policy-Pilot internal overview. Generated from the live repository on 17 August 2026. This document summarizes architecture; `AGENTS.md` and `.cursor/rules/` remain the operational source of truth for agents.